Understanding ACT: Core Architecture and Data Pipeline

Explaining Action Chunking with Transformers

1 Introduction

Action Chunking with Transformers (ACT) is an imitation learning algorithm designed for high-precision, closed-loop manipulation. The core idea is to predict sequences of actions (action chunks) instead of a single step, significantly reducing the effective horizon of tasks and mitigating compounding errors that typically plague behavioral cloning in fine manipulation.

While the original ACT paper focused heavily on a bimanual setup (14 DoF total) with a fixed 4-camera configuration, the architecture is highly modular. In our implementation, we tailor the pipeline for a **single robotic arm** utilizing a flexible **2 to 3 camera setup**.

At a high level, the ACT policy behaves as a Conditional Variational Autoencoder (CVAE) decoder at inference time. It takes in real-time camera feeds, the robot's proprioceptive state (joint positions), and a "style variable" z (sampled from a prior), and outputs an action sequence. The bulk of this modeling relies on a Convolutional Neural Network (CNN) backbone coupled with a Transformer Encoder-Decoder architecture.

2 The Input Pipeline: Preparing the Sequence

Before the raw data from the robot's hardware is tokenized, it must undergo a crucial normalization step.

2.1 Data Normalization

Neural networks require data to be standardized to ensure numerical stability and prevent attributes with large numerical ranges (like rotary motors) from overpowering attributes with small ranges (like gripper apertures).

- **Vision:** Raw pixel values are typically rescaled from integers $[0, 255]$ to floats $[0, 1]$. To match the distribution expected by the pretrained ResNet backbone, they are further normalized by subtracting the distribution mean and dividing by the standard deviation. This centers the color channels around zero, stabilizing the math inside the deep convolutional layers.
- **Proprioception & Actions:** The raw physical joint angles and actions are normalized using `MEAN_STD` statistics calculated across the entire training dataset. Each value is transformed using the formula $x_{norm} = \frac{x_{raw} - \mu}{\sigma}$. This ensures every joint dimension exists on a standard scale (roughly between -2 and 2). During deployment, the policy's final action predictions are automatically un-normalized back into raw physical commands before being sent to the hardware.

2.2 Vision Data Processing

The robot's visual observations (incoming from 2 or 3 RGB cameras) form the heaviest part of the input. Each camera stream is processed independently but symmetrically:

- **CNN Backbone (ResNet-18):** Each $480 \times 640 \times 3$ RGB image is passed through a ResNet-18 backbone. Rather than compressing the image into a single vector, the final spatial pooling layer of the ResNet is omitted. This preserves a spatial feature map of shape $15 \times 20 \times 512$ per camera.
- **Flattening:** This spatial grid is flattened into a sequence of 300 vectors ($15 \times 20 = 300$), each of dimension 512. This transforms the grid-like image down into a token sequence that a Transformer can process.
- **2D Positional Embeddings:** Since flattening inherently destroys the structural 2D relationship between patches (e.g., that patch 1 is directly above patch 16), a **2D sinusoidal positional embedding** is added to the feature sequence. This allows the Transformer to retain a spatial understanding of the image.
- **Concatenation:** The sequences from all active cameras are simply concatenated end-to-end.
  - For a **2-camera** setup (e.g., wrist and top view), the total visual sequence length is $2 \times 300 =$ **600 tokens**.
  - For a **3-camera** setup, the sequence expands to $3 \times 300 =$ **900 tokens**.

Simple Summary: To recap the vision pipeline, each raw $480 \times 640 \times 3$ RGB image is transformed by the ResNet into a 15×20 grid of feature vectors (512 coordinates). We then generate a matching 15×20 grid of 2D positional embeddings (also 512 coordinates) and **add** them element-wise to the features. Finally, this grid is flattened into 300 unique vectors of dimension 512. In these vectors, the visual information and the spatial coordinates are mixed in a subtle way that allow the Transformer to understand exactly *what* it is seeing and *where* it is in the 2D frame.

2.3 Proprioception Processing (Robot State)

Unlike vision, the physical state of the robot is exceptionally low-dimensional. For our single-arm setup, the raw proprioceptive input is just the absolute joint positions of that single arm (e.g., a 6-dimensional vector).

- **Projection:** To allow the Transformer to mix the robot's state with the high-dimensional visual features, this small joint position vector is passed through a single Multilayer Perceptron (MLP) or linear layer that maps it up to the Transformer's hidden dimension of **512**.
- **State Token:** This results in exactly **1 token** representing the entire physical posture of the arm at that exact moment.

Simple Summary: To recap the proprioception pipeline, the raw joint positions of the robot arm are passed through a linear layer that maps them to 512-dimensional vectors. This results in exactly 1 token representing the entire physical posture of the arm at that exact moment.